

Document de Conception

Compilateur Deca

Projet GL51

Équipe : Thibault, Akram, Ahmad, Ikhlass, Widad

Institution : Ensimag (Grenoble INP)

Date : Janvier 2026

Table des matières

1	Architecture Globale	3
1.1	Ajouts sur l'Architecture	3
1.2	Patron de Conception : Interprète (Pattern Interpreter)	3
1.3	Flux de Compilation dans DecacCompiler	3
1.4	Passage d'État Sans Variables Globales	4
2	Étape A : Analyse Syntaxique	4
2.1	Spécifications sur l'analyse lexicale et syntaxique	4
2.1.1	Analyse Lexicale	4
2.1.2	Analyse Syntaxique	5
2.2	Utilisation d'ANTLR4	5
2.3	Conversion Arbre ANTLR → Arbre Abstrait (AST)	5
2.4	Gestion des Inclusions (#include)	6
3	Étape B : Vérifications Contextuelles	6
3.1	Spécifications et Vérifications	6
3.1.1	Compatibilité de Type	6
3.1.2	La Déclaration du Type Object	6
3.2	Description des algorithmes et structures de données employés	7
3.2.1	Gestion des environnements et des définitions	7
3.2.2	Construction de l'arbre de syntaxe pour les structures conditionnelles	7
3.3	Structure des Environnements (Détails)	7
3.3.1	EnvironmentExp (Variables et Méthodes)	7
3.3.2	EnvironmentType (Types et Classes)	7
3.4	Les 3 Passes de Vérification	8
3.4.1	Passé 1 : Découverte des Classes (verifyListClass)	8
3.4.2	Passé 2 : Signatures des Membres (verifyListClassMembers)	8
3.4.3	Passé 3 : Corps et Initialisations	8
3.5	Décoration de l'Arbre	9
4	Étape C : Génération de Code	9
4.1	Optimisations lors de la génération de code	9
4.2	Gestion de la Mémoire	9
4.2.1	Zone Globale (GB - Global Base)	10
4.2.2	Pile (LB/SP - Local Base / Stack Pointer)	10
4.2.3	Tas (Heap)	10
4.3	Table des Méthodes Virtuelles (VTable)	11
4.3.1	Construction de la VTable (Passé 1 de codeGenProgram)	11
4.3.2	Appel de Méthode Virtuelle (MethodCall)	11
4.4	Allocation des Registres	11
4.4.1	Registres Scratch (R0, R1)	12
4.4.2	Registres Sauvegardés (R2-R15)	12
4.4.3	Gestion du Débordement (Spill)	12
4.4.4	Reset des Registres	12
4.5	Gestion des Erreurs Runtime	13
4.5.1	Mécanisme de Déclenchement	13
4.5.2	Vecteurs d'Interruption Supportés	13
5	Diagramme d'Architecture CodeGen	14

6	Points Critiques et Limitations	14
6.1	Allocation de Registres Non-Robuste	14
6.2	Optimisations Non Implémentées	15
6.3	Gestion de la Pile Simplifiée	15
7	Conclusion	15

1 Architecture Globale

1.1 Ajouts sur l'Architecture

La majorité des ajouts a été réalisée lors de la partie relative aux objets. L'implémentation repose sur l'architecture de base fournie, en particulier sur l'utilisation d'une classe abstraite suivie de classes concrètes, conformément au modèle initial du compilateur. Cette approche a permis de conserver une cohérence globale avec l'existant tout en facilitant l'extension des fonctionnalités.

Par ailleurs, plusieurs classes d'erreurs spécifiques ont été ajoutées afin de gérer les cas particuliers rencontrés lors de l'analyse syntaxique. Ces classes sont intégrées au parseur et permettent une gestion plus fine et plus explicite des erreurs, sans modifier en profondeur les mécanismes déjà en place.

Dans l'ensemble, nous avons cherché à réutiliser au maximum le code existant, en ne nous en écartant que lorsque cela était strictement nécessaire pour répondre aux nouvelles exigences.

1.2 Patron de Conception : Interprète (Pattern Interpreter)

Le compilateur Deca utilise le Patron Interprète pour représenter et traiter le programme source sous forme d'un Arbre de Syntaxe Abstraite (AST).

Classes abstraites principales :

- **Tree** : Racine de toute la hiérarchie. Fournit les mécanismes de base (location, decompile, prettyPrint).
- **AbstractExpr** : Représente toute expression ayant une valeur. Hérite de **AbstractInst**.
- **AbstractInst** : Représente toute instruction (affectation, boucle, etc.).
- **AbstractProgram**, **AbstractMain**, **AbstractDeclClass** : Racines pour les structures de haut niveau.

Chaque nœud concret (exemple : **Plus**, **While**, **MethodCall**) hérite de ces abstractions et redéfinit les méthodes de traitement :

- **decompile()** : Régénération du code source
- **verify***() : Vérifications contextuelles (typage, portées)
- **codeGen***() : Génération de code assembleur IMA

1.3 Flux de Compilation dans DecacCompiler

La classe **DecacCompiler** orchestre les 3 phases principales. Elle est instanciée une fois par fichier source :

```
1 DecacCompiler.compile()  
2 |-> doLexingAndParsing() : Phase A (Analyse Syntaxique)  
3 |-> prog.verifyProgram() : Phase B (Verifications Contextuelles)  
4 |-> prog.codeGenProgram() : Phase C (Generation de Code)  
5
```

Listing 1 – Flux de compilation

Enchaînement détaillé :

Phase A Construction de l'AST via ANTLR4

- Lexer → Parser → Construction des nœuds **Tree**
- Retourne un **AbstractProgram** représentant l'arbre complet

Phase B Vérifications contextuelles en 3 passes

1. `verifyListClass()` : Découverte des classes et hiérarchie d'héritage
2. `verifyListClassMembers()` : Signatures des champs et méthodes
3. `verifyListClassBody()` : Vérification des corps de méthodes et initialisations
4. `verifyMain()` : Vérification du bloc principal

Phase C Génération de code IMA

- Construction des VTables (tables de méthodes virtuelles)
- Code d'initialisation des objets
- Code des méthodes
- Code du programme principal
- Routines de service d'interruption (ISR)

1.4 Passage d'État Sans Variables Globales

Design Anti-Static : Le compilateur évite les variables globales/statiques en passant explicitement l'instance `DecacCompiler` à chaque méthode de l'arbre.

Composants encapsulés dans DecacCompiler :

- `SymbolTable` : Table de symboles globale (internement des identifiants)
- `EnvironmentType` : Environnement des types prédéfinis et classes
- `RegisterManager` : Allocation des registres R2-R15
- `MemoryManagementUnit` : Gestion pile/tas/globales
- `InterruptController` : Gestion des erreurs runtime (BOV, null pointer, etc.)
- `IMAProgram` : Buffer d'accumulation des instructions assembleur

Mécanisme de propagation :

```

1 // Chaque noeud de l'arbre recoit le compiler
2 public abstract Type verifyExpr(DecacCompiler compiler,
3                                 EnvironmentExp localEnv,
4                                 ClassDefinition currentClass);
5
6 protected abstract void codeGenExpr(DecacCompiler compiler,
7                                     GPRegister register);
8

```

Listing 2 – Propagation du contexte de compilation

Cela permet :

- Isolation : Chaque compilation est indépendante (mode parallèle -P)
- Testabilité : Possibilité de créer des instances de test
- Traçabilité : Accès aux métadonnées (source file, options) depuis n'importe quel nœud

2 Étape A : Analyse Syntaxique

2.1 Spécifications sur l'analyse lexicale et syntaxique

2.1.1 Analyse Lexicale

L'implémentation de l'analyse lexicale et syntaxique repose principalement sur la conversion de la lexicographie Deca vers ANTLR4. Cette étape ne nécessite pas de modifications conceptuelles majeures, mais demande une bonne compréhension de la traduction des expressions lexicales. Par exemple, certains opérateurs et motifs doivent être adaptés à la syntaxe propre à ANTLR, comme la transformation de `+` en `|` ou `('' + 'e'+'f')` en `[ef]`?

2.1.2 Analyse Syntaxique

Du côté du parseur, la principale difficulté réside dans la création des classes manquantes lorsque celles-ci ne sont pas fournies par l'architecture de base. Ce travail nécessite une correspondance rigoureuse entre la grammaire et la structure des objets manipulés par le compilateur, ainsi qu'une attention particulière portée à la détection et à la gestion des erreurs.

Un point particulièrement important concerne la gestion des informations de localisation. Il est impératif d'appeler la méthode `setLocation` pour l'ensemble des nœuds de l'arbre de syntaxe abstraite. Ces informations sont indispensables pour produire des messages d'erreur précis et conformes aux attentes. Un oubli de localisation peut entraîner des comportements incorrects lors de l'exécution des tests : par exemple, un programme erroné peut être considéré comme valide ou provoquer un arrêt prématuré du compilateur avant que l'erreur ne soit correctement signalée.

2.2 Utilisation d'ANTLR4

Le compilateur utilise ANTLR4 pour générer automatiquement le Lexer et le Parser à partir de grammaires déclaratives.

Fichiers grammaire :

- `DecaLexer.g4` : Définit les tokens (IDENT, INT, FLOAT, mots-clés, opérateurs)
- `DecaParser.g4` : Définit les règles de syntaxe du langage Deca

Génération des classes Java :

```
1 src/main/antlr4/fr/ensimag/deca/syntax/*.g4
2     (Maven plugin ANTLR4)
3 target/generated-sources/antlr4/*.java
4
```

Classes générées :

- `DecaLexer.java` : Analyse lexicale (stream de caractères → stream de tokens)
- `DecaParser.java` : Analyse syntaxique (stream de tokens → arbre ANTLR)

2.3 Conversion Arbre ANTLR → Arbre Abstrait (AST)

L'arbre ANTLR (`ParseTree`) est immédiatement converti en arbre abstrait (`Tree`) pendant le parsing via les actions embarquées dans `DecaParser.g4`.

Mécanisme :

```
1 prog returns[AbstractProgram tree]
2 : list_classes main EOF {
3     $tree = new Program($list_classes.tree, $main.tree);
4     setLocation($tree, $list_classes.start);
5 }
6 ;
7
```

Listing 3 – Exemple de règle ANTLR avec action

Avantages :

- Pas de visite séparée de l'arbre ANTLR
- Construction directe des nœuds métier (`Program`, `DeclClass`, `Plus`, etc.)
- Annotation immédiate de la Location (ligne, colonne) pour les messages d'erreur

Classes support :

- **AbstractDecaParser** : Superclasse du parser généré, contient les helpers (`setLocation()`, gestion d'erreurs)
- **AbstractDecaLexer** : Superclasse du lexer, gère les includes

2.4 Gestion des Inclusions (`#include`)

Le lexer supporte la directive `#include "fichier.decah"` pour inclure des fichiers header (bibliothèque Math, etc.).

Implémentation : Pile de streams dans **AbstractDecaLexer**

1. Lorsqu'un `#include` est rencontré, le lexer :
2. Sauvegarde l'état courant (`CharStream`, position) dans une pile
3. Ouvre le fichier inclus et bascule dessus
4. À la fin du fichier inclus, restaure l'état précédent

Protection contre les inclusions circulaires :

- Détection via une `Set<String>` de fichiers déjà inclus
- Exception `CircularInclude` levée si boucle détectée

Gestion d'erreurs spécifiques :

- `IncludeFileNotFound` : Fichier introuvable
- Chemin de recherche : répertoire courant, puis `src/main/resources/include/`

3 Étape B : Vérifications Contextuelles

3.1 Spécifications et Vérifications

3.1.1 Compatibilité de Type

Dans le cadre de la partie B, passe 3, des fonctions spécifiques ont été ajoutées à la classe **EnvironnementType** afin de gérer les vérifications de compatibilité des types. Ces fonctions regroupent l'ensemble des conditions nécessaires à cette passe et permettent de centraliser la logique de vérification.

Ce choix de conception vise à éviter la duplication de code et à améliorer la réutilisabilité des mécanismes de vérification, notamment lors de leur utilisation dans d'autres parties du compilateur.

3.1.2 La Déclaration du Type Object

Concernant l'implémentation du type **Object**, l'essentiel du travail s'est concentré sur la création et l'initialisation de l'environnement de types. Cela comprend :

- la déclaration du type **Object**.
- l'ajout de la méthode `equals` dans l'environnement dès son initialisation.

Cependant, l'implémentation effective de la méthode `equals` n'est pas réalisée de manière classique dans le code source. L'idée initiale était d'ajouter le code correspondant à cette méthode à la fin de la compilation, directement dans le programme généré. Cette approche n'a toutefois pas pu être finalisée dans le rendu final.

3.2 Description des algorithmes et structures de données employés

Les structures de données existantes ont été largement réutilisées dans l'ensemble du programme. En particulier, les structures de type **TreeList** sont employées dans la majorité des composants. Ce choix permet de conserver une représentation homogène des arbres de syntaxe et de limiter les modifications apportées à l'architecture initiale du compilateur.

3.2.1 Gestion des environnements et des définitions

Dans la classe **EnvironnementExp**, des tables de hachage (**HashMap**) ont été introduites pour la gestion des définitions. Ce choix est motivé par le besoin de retrouver rapidement un objet à partir de sa clé et de vérifier efficacement l'existence d'une définition. Ces deux opérations étant réalisées en temps moyen constant, l'utilisation d'une **HashMap** s'est révélée particulièrement adaptée dans ce contexte.

3.2.2 Construction de l'arbre de syntaxe pour les structures conditionnelles

Un algorithme spécifique a été introduit au niveau du parseur pour la gestion des constructions **if-then-else**. La grammaire fournie ne générant pas ces structures de manière récursive, la construction correcte de l'arbre de syntaxe abstraite nécessite de connaître l'instruction suivante. Pour répondre à cette contrainte, les blocs **if-then-else** sont temporairement stockés dans une **ArrayList**, à l'aide d'une classe dédiée. Cette structure intermédiaire permet ensuite de reconstruire l'arbre conformément aux spécifications attendues.

3.3 Structure des Environnements (Détails)

Le compilateur utilise deux types d'environnements pour gérer les portées :

3.3.1 EnvironmentExp (Variables et Méthodes)

Structure : Liste chaînée de dictionnaires **HashMap<Symbol, ExpDefinition>**

- Chaque dictionnaire représente une portée (bloc, classe, méthode)
- Pointeur **parentEnvironment** vers la portée englobante

Algorithme de recherche (get(Symbol)) :

1. Recherche dans le dictionnaire courant
2. Si échec, recherche récursive dans **parentEnvironment**
3. Retourne **null** si introuvable

Algorithme d'insertion (declare(Symbol, ExpDefinition)) :

1. Vérifie que le symbole n'existe pas déjà dans le dictionnaire courant
2. Lève **DoubleDefException** si redéclaration locale
3. Sinon, insertion dans le dictionnaire courant (masquage autorisé des portées parentes)

3.3.2 EnvironmentType (Types et Classes)

Structure : Dictionnaire plat **HashMap<Symbol, TypeDefinition>**

- Contient les types prédéfinis (**int**, **float**, **boolean**, **void**, **null**, **Object**)
- Enrichi dynamiquement avec les classes déclarées (**declareClass()**)

Hiérarchie d'héritage :

- Chaque `ClassDefinition` contient un pointeur vers sa superclasse
- La classe racine `Object` est prédéfinie dans `EnvironmentType`

3.4 Les 3 Passes de Vérification

Le processus de vérification est strictement séquentiel pour respecter les dépendances.

3.4.1 Passe 1 : Découverte des Classes (`verifyListClass`)

Objectif : Construire la hiérarchie des classes sans analyser les membres

Actions pour chaque classe :

1. Vérifier que la superclasse existe et est bien une classe
2. Créer un `ClassType` avec pointeur vers la `ClassDefinition` de la superclasse
3. Enregistrer dans `EnvironmentType` global
4. Créer un `EnvironmentExp` vide pour les membres (rempli en Passe 2)

Détection de cycles d'héritage :

- Remontée de la chaîne `superClassDef` jusqu'à `Object`
- Exception si on retombe sur la classe de départ

3.4.2 Passe 2 : Signatures des Membres (`verifyListClassMembers`)

Objectif : Déclarer champs et méthodes sans vérifier les corps

Pour chaque champ :

1. Vérifier le type
2. Créer `FieldDefinition` avec index (position dans l'objet)
3. Insérer dans `EnvironmentExp` de la classe

Pour chaque méthode :

1. Vérifier le type de retour et la signature (types des paramètres)
2. Si redéfinition (override), vérifier compatibilité avec la méthode héritée :
 - Même nombre de paramètres
 - Types des paramètres identiques
 - Type de retour covariant (sous-type autorisé)
3. Créer `MethodDefinition` avec index dans la `VTable`
4. Générer un Label unique : `code.<classe>.<méthode>`
5. Insérer dans `EnvironmentExp` de la classe

3.4.3 Passe 3 : Corps et Initialisations

Objectif : Vérifier les expressions et instructions

Pour chaque initialisation de champ :

- Vérifier que l'expression est compatible avec le type du champ
- Insertion de `ConvFloat` implicite si nécessaire

Pour chaque corps de méthode :

- Créer un nouvel `EnvironmentExp` avec les paramètres
- Vérifier chaque instruction (`verifyInst`)
- Vérifier que les chemins de retour sont cohérents avec le type de retour

Pour le bloc principal :

- Pas de classe courante (`currentClass = null`)
- Pas de type de retour attendu (`returnType = void`)

3.5 Décoration de l'Arbre

Pendant la phase B, l'arbre est décoré avec des informations sémantiques :

Type Decoration : Chaque `AbstractExpr` stocke son `Type`

```
1 private Type type; // Set par verifyExpr()
2 public Type getType() { return type; }
3
```

Definition Decoration : Chaque `AbstractIdentifier` stocke sa `Definition`

```
1 private Definition definition; // FieldDefinition, MethodDefinition, etc.
2 public Definition getDefinition() { return definition; }
3
```

Nœuds implicites : ConvFloat

- Lorsqu'un `int` est affecté à un `float`, un nœud `ConvFloat` est inséré dans l'arbre
- Exemple : `float x = 5;` → AST devient `float x = ConvFloat(IntLiteral(5));`
- La conversion sera traduite par l'instruction `FLOAT` en phase C

Vérification de la décoration : Assertion `checkAllDecorations()` avant la génération de code

4 Étape C : Génération de Code**4.1 Optimisations lors de la génération de code**

Dans la phase de génération de code (*codegen*), une optimisation a été implémentée pour les opérations de multiplication et de division. Lorsqu'un littéral correspondant à une puissance de deux est détecté, l'opération arithmétique est remplacée par une instruction de décalage (*shift*). Cette transformation permet, en théorie, d'améliorer significativement les performances d'exécution dans ces cas particuliers, tout en restant transparente pour le reste du compilateur.

4.2 Gestion de la Mémoire

Le compilateur IMA utilise 3 zones mémoires distinctes.

4.2.1 Zone Globale (GB - Global Base)

Usage : Variables globales et VTables

- Adressage : `RegisterOffset(offset, Register.GB)`
- Allocation : `MemoryManagementUnit.allocGlobal(sizeWords)`
- Première adresse : `1(GB)` (car `0(GB)` réservé pour `null`)

Contenu typique :

```
1 ; 1(GB) = VTable de Object
2 ; 2(GB) = VTable de MaClasse
3 ; 3(GB) = Variable globale x
4
```

4.2.2 Pile (LB/SP - Local Base / Stack Pointer)

Usage : Variables locales, paramètres, sauvegarde de contexte

LB (Local Base) : Pointe vers le début de la frame de la méthode courante

- Variables locales : `1(LB)`, `2(LB)`, ...
- Paramètres : `-3(LB)` (premier paramètre), `-4(LB)`, ... (ordre inverse)
- Adresse de retour : `0(LB)`
- LB précédent : `-1(LB)`

SP (Stack Pointer) : Pointe vers le sommet de la pile

- Utilisé pour l'empilement temporaire (évaluation d'expressions, appels de méthodes)

Instruction TSTO : Vérifie qu'il reste assez de place avant d'entrer dans une méthode

```
1 TSTO #maxStack
2 BOV stack_overflow_error
3
```

Calcul de TSTO : `maxStack = nb_variables_locales + max(nb_temporaires_simultanés)`

- La MMU trace dynamiquement les `notifyPush()`/`notifyPop()` pour calculer le high-water mark

4.2.3 Tas (Heap)

Usage : Allocation d'objets (`new`)

Instruction NEW :

```
1 NEW #taille, Rn ; Rn = adresse du nouvel objet
2 BOV heap_overflow_error
3
```

Structure d'un objet en mémoire :

```
1 Offset 0 : Adresse de la VTable
2 Offset 1 : Premier champ
3 Offset 2 : Deuxieme champ
4 ...
5
```

4.3 Table des Méthodes Virtuelles (VTable)

La VTable permet le dispatch dynamique (polymorphisme) en stockant les adresses des méthodes d'une classe.

4.3.1 Construction de la VTable (Passe 1 de codeGenProgram)

Pour chaque classe :

1. Allouer un espace en zone globale : `allocGlobal(nb_méthodes + 1)`
 - Slot 0 : Adresse de la VTable de la superclasse (ou `null` pour `Object`)
 - Slot 1..n : Adresses des méthodes
2. Si héritage, copier les entrées de la VTable de la superclasse

```
1 LOAD superVTable(GB), R0
2 STORE R0, classVTable(GB) ; Copie des pointeurs
3
```

3. Pour chaque méthode définie ou redéfinie dans la classe courante :
 - Générer le label : `code.<classe>.<méthode>`
 - Stocker l'adresse dans la VTable à l'index de la méthode

```
1 LOAD #code.MaClasse.maMethode, R0
2 STORE R0, classVTable+index(GB)
3
```

Index des méthodes :

- Hérité de la superclasse : même index que dans la VTable parente (override)
- Nouvelle méthode : `index = nombre de méthodes de la superclasse + position`

4.3.2 Appel de Méthode Virtuelle (MethodCall)

Algorithme :

1. Évaluer l'expression objet $\rightarrow R_n$ (adresse de l'objet)
2. Vérification null : `CMP #null, Rn + BOV null_pointer_error`
3. Empiler les arguments (ordre inverse) + `this`

```
1 ; Arguments: arg1, arg2
2 <eval arg2> ; -> R0
3 PUSH R0
4 <eval arg1> ; -> R0
5 PUSH R0
6 PUSH Rn ; this
7
```

4. Charger la VTable : `LOAD 0(Rn), R0` (slot 0 de l'objet)
5. Charger l'adresse de la méthode : `LOAD index(R0), R0`
6. Appel indirect : `BSR R0`
7. Récupérer la valeur de retour : Résultat dans `R0`
8. Dépiler : `SUBSP #(nb_args + 1)`

4.4 Allocation des Registres

Le `RegisterManager` gère les registres `R2` à `R15` (`R0/R1` sont scratch - usage temporaire).

4.4.1 Registres Scratch (R0, R1)

- R0 : Utilisé librement pour calculs intermédiaires, chargements, résultats de fonction
- R1 : Réserve pour FMA (opérande implicite) et opérations secondaires
- Jamais alloués par `prendreRegistre()`

4.4.2 Registres Sauvegardés (R2-R15)

Algorithme d'allocation : Simple compteur incrémental (`registreCourant`)

`prendreRegistre()` :

1. Vérifier `registreCourant <= registreMax`
2. Si pas de registre libre : Exception `DecacInternalError` (pas de spill automatique)
3. Retourner `Register.getR(registreCourant)` et incrémenter

`libererRegistre()` :

1. Décrémenter `registreCourant`
2. Sauvegarde : ne jamais descendre sous 2

Limite configurable : Option `-r X` (défaut 16)

- Permet de tester le comportement avec peu de registres
- Minimum : 4 (R0, R1, R2, R3)

4.4.3 Gestion du Débordement (Spill)

Problème : Que faire si plus de registres disponibles lors d'une expression complexe ?

Solution actuelle : Pas de spill automatique

- Le compilateur lève une `DecacInternalError`
- Responsabilité : Les méthodes `codeGenExpr` doivent libérer les registres après usage

Pattern d'utilisation :

```

1 GPRRegister reg = compiler.getRegisterManager().prendreRegistre();
2 leftOperand.codeGenExpr(compiler, reg); // Evaluation dans reg
3 // Utiliser reg
4 compiler.getRegisterManager().libererRegistre();
5
```

Alternative envisageable (non implémentée) :

- Sauvegarder les registres débordants sur la pile (PUSH)
- Restaurer à la libération (POP)
- Nécessiterait un tracking dans la MMU

4.4.4 Reset des Registres

Méthode `reset()` : Réinitialise `registreCourant = 2`

- Appelée au début de chaque méthode (`codeGenDeclMethod`)
- Garantit l'isolation entre méthodes

4.5 Gestion des Erreurs Runtime

Le `InterruptController` génère les routines de service pour les exceptions runtime.

4.5.1 Mécanisme de Déclenchement

Étape 1 : Armement (pendant la génération de code)

```
1 compiler.getIrqController().triggerInterrupt(compiler,
2     InterruptVector.IRQ_NULL_PTR);
3
```

- Ajoute un BOV `<label>` après l'instruction critique
- Enregistre l'interruption dans `Set<InterruptVector> armedInterrupts`

Étape 2 : Génération des ISR (à la fin de `codeGenProgram`)

```
1 compiler.getIrqController().flashServiceRoutines(compiler);
2
```

Pour chaque interruption armée, génère :

```
1 <label>:
2     WSTR "<message d'erreur>"
3     WNL
4     ERROR
5
```

4.5.2 Vecteurs d'Interruption Supportés

IRQ_STACK_OVERFLOW Débordement de pile

- Déclenché après `TSTO`
- Message : "Error : Stack overflow"

IRQ_HEAP_FULL Tas plein

- Déclenché après `NEW`
- Message : "Error : Heap full"

IRQ_NULL_PTR Déréférencement de null

- Déclenché avant accès champ/méthode
- Message : "Error : Null pointer dereference"

IRQ_DIVIDE_BY_ZERO Division par zéro

- Déclenché après `DIV` ou `REM`
- Message : "Error : Division by zero"

IRQ_INVALID_CAST Cast illégal

- Déclenché après vérification de `instanceof`
- Message : "Error : Invalid cast"

IRQ_MISSING_RET Retour manquant

- Déclenché à la fin d'une méthode non-void sans `return`
- Message : "Error : Missing return statement"

Option -no-check : Désactive toutes les vérifications (optimisation)

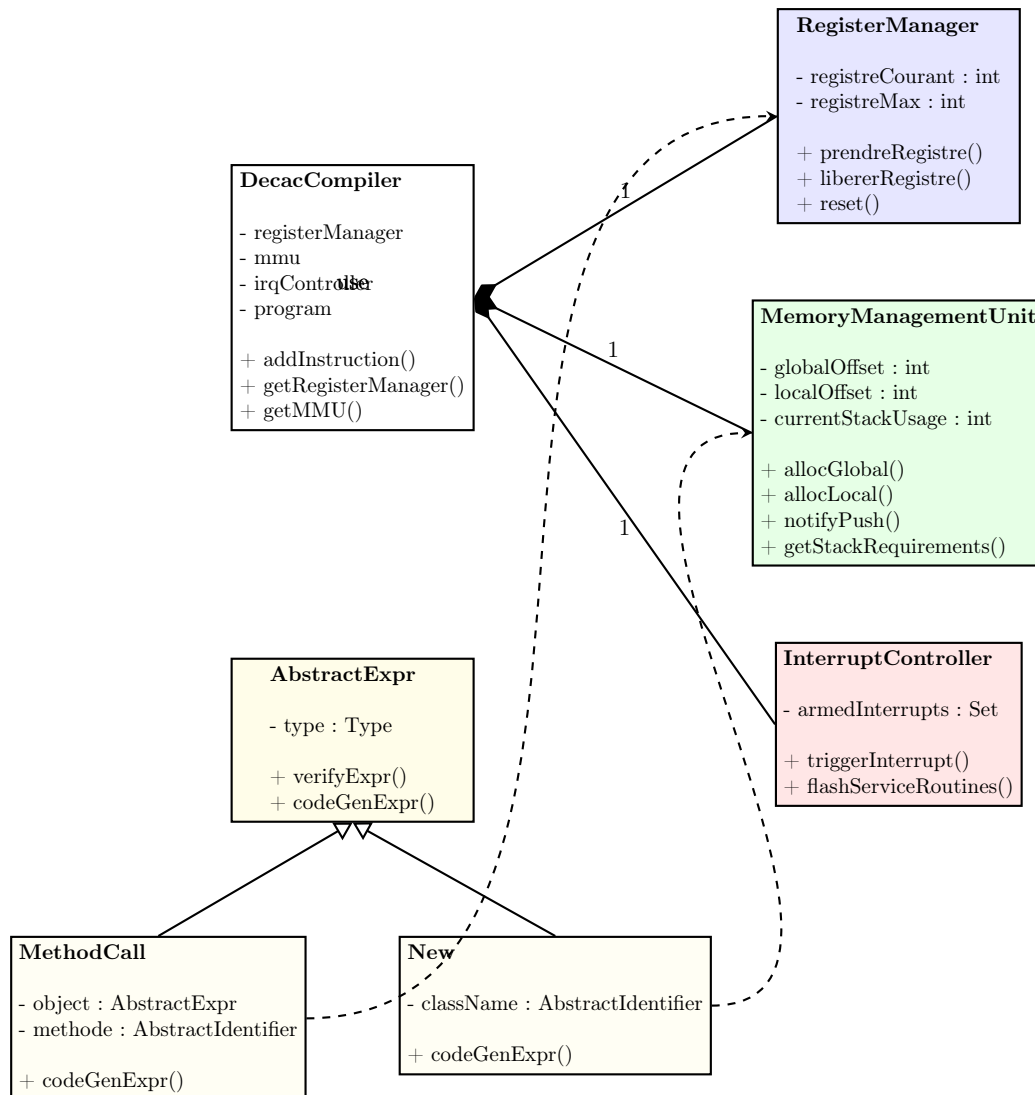


FIGURE 1 – Architecture des composants de génération de code

5 Diagramme d'Architecture CodeGen

Relations entre composants :

- DecacCompiler possède (composition $\diamond$) un RegisterManager, une MemoryManagementUnit et un InterruptController
- MethodCall et New héritent ($\triangleright$) de AbstractExpr
- MethodCall et New utilisent (dépendance $\cdots >$) les services du RegisterManager et de la MMU

6 Points Critiques et Limitations

6.1 Allocation de Registres Non-Robuste

- Absence de spill automatique : Le compilateur plante si une expression complexe nécessite plus de registres que disponibles
- Solution temporaire : Structurer les expressions pour minimiser l'usage simultané de registres

6.2 Optimisations Non Implémentées

- Pas d'allocation intelligente (toujours ordre linéaire $R2 \rightarrow R15$)
- Pas de réutilisation de registres morts (liveness analysis)
- Pas d'inlining de méthodes simples

6.3 Gestion de la Pile Simplifiée

- TSTO calculé de manière conservative (pire cas)
- Pas d'optimisation des frames d'appel (élimination de LB pour méthodes leaf)

7 Conclusion

Ce document décrit l'architecture interne du compilateur Deca, en mettant l'accent sur les décisions de conception et les mécanismes réels implémentés. Le compilateur suit une architecture classique multi-passes avec une séparation claire des responsabilités :

- Analyse syntaxique : ANTLR4 avec conversion directe en AST
- Vérifications contextuelles : 3 passes avec environnements chaînés et décoration de l'arbre
- Génération de code : Pattern Interpreter avec gestion explicite de la mémoire et des registres

Les points d'amélioration identifiés (spill, optimisations) constituent des extensions naturelles pour un développement futur.